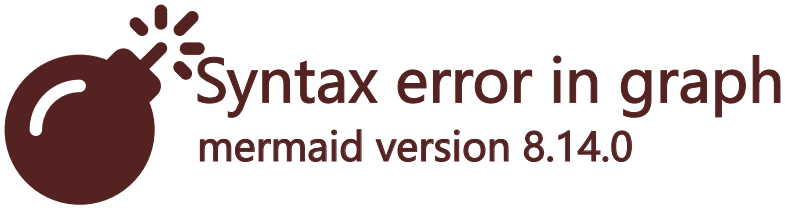
第二节 数据准备模块实现

RAG系统的效果很大程度上取决于数据准备的质量。在上一节中，我们明确了"小块检索，大块生成"的父子文本块策略。接下来学习如何将数据准备部分的架构思想转化为可运行的代码。



一、核心设计

数据准备模块的核心是实现"小块检索，大块生成"的父子文本块架构。

父子文本块映射关系：

```
父文档（完整菜谱）
├── 子块1：菜品介绍 + 难度评级
├── 子块2：必备原料和工具
├── 子块3：计算（用量配比）
├── 子块4：操作（制作步骤）
└── 子块5：附加内容（变化做法）
```

基本流程：

- **检索阶段**：使用小的子块进行精确匹配，提高检索准确性
- **生成阶段**：传递完整的父文档给LLM，确保上下文完整性
- **智能去重**：当检索到同一道菜的多个子块时，合并为一个完整菜谱

元数据增强：

- **菜品分类**：从文件路径推断（荤菜、素菜、汤品等）
- **难度等级**：从内容中的星级标记提取
- **菜品名称**：从文件名提取
- **文档关系**：建立父子文档的ID映射关系

二、模块实现详解

> data_preparation.py完整代码

2.1 类结构设计

```python
class DataPreparationModule:
    """数据准备模块 - 负责数据加载、清洗和预处理"""

    def __init__(self, data_path: str):
        self.data_path = data_path
        self.documents: List[Document] = []  # 父文档（完整食谱）
        self.chunks: List[Document] = []     # 子文档（按标题分割的小块）
        self.parent_child_map: Dict[str, str] = {}  # 子块ID -> 父文档ID的映射
```

- `documents`：存储完整的菜谱文档（父文档）
- `chunks`：存储按标题分割的小块（子文档）
- `parent_child_map`：维护父子关系映射

2.2 文档加载实现

2.2.1 批量加载Markdown文件

```
def load_documents(self) -> List[Document]:
    """加载文档数据"""
    documents = []
    data_path_obj = Path(self.data_path)

    for md_file in data_path_obj.rglob("*.md"):
        # 读取文件内容，保持Markdown格式
        with open(md_file, 'r', encoding='utf-8') as f:
            content = f.read()

        # 为每个父文档分配唯一ID
        parent_id = str(uuid.uuid4())

        # 创建Document对象
        doc = Document(
            page_content=content,
            metadata={
                "source": str(md_file),
                "parent_id": parent_id,
                "doc_type": "parent"  # 标记为父文档
            }
        )
        documents.append(doc)

    # 增强文档元数据
    for doc in documents:
        self._enhance_metadata(doc)

    self.documents = documents
    return documents
```

- `rglob("*.md")`：递归查找所有Markdown文件
- `parent_id`：为每个父文档分配唯一ID，建立父子关系的关键
- `doc_type`：标记为"parent"，便于区分父子文档

2.2.2 元数据增强

```
def _enhance_metadata(self, doc: Document):
    """增强文档元数据"""
    file_path = Path(doc.metadata.get('source', ''))
    path_parts = file_path.parts

    # 提取菜品分类
    category_mapping = {
        'meat_dish': '荤菜', 'vegetable_dish': '素菜', 'soup': '汤品',
        'dessert': '甜品', 'breakfast': '早餐', 'staple': '主食',
```

```
        'aquatic': '水产', 'condiment': '调料', 'drink': '饮品'
    }

    # 从文件路径推断分类
    doc.metadata['category'] = '其他'
    for key, value in category_mapping.items():
        if key in file_path.parts:
            doc.metadata['category'] = value
            break

    # 提取菜品名称
    doc.metadata['dish_name'] = file_path.stem

    # 分析难度等级
    content = doc.page_content
    if '★★★★★' in content:
        doc.metadata['difficulty'] = '非常困难'
    elif '★★★★' in content:
        doc.metadata['difficulty'] = '困难'
    # ... (其他难度等级判断)
```

- **分类推断**: 从HowToCook项目的目录结构推断菜品分类
- **难度提取**: 从内容中的星级标记自动提取难度等级
- **名称提取**: 直接使用文件名作为菜品名称

2.3 Markdown结构分块

将完整的菜谱文档按照Markdown标题结构进行分块，实现父子文本块架构。

2.3.1 分块策略

```
def chunk_documents(self) -> List[Document]:
    """Markdown结构感知分块"""
    if not self.documents:
        raise ValueError("请先加载文档")

    # 使用Markdown标题分割器
    chunks = self._markdown_header_split()

    # 为每个chunk添加基础元数据
    for i, chunk in enumerate(chunks):
        if 'chunk_id' not in chunk.metadata:
            # 如果没有chunk_id（比如分割失败的情况），则生成一个
            chunk.metadata['chunk_id'] = str(uuid.uuid4())
        chunk.metadata['batch_index'] = i  # 在当前批次中的索引
        chunk.metadata['chunk_size'] = len(chunk.page_content)

    self.chunks = chunks
    return chunks
```

2.3.2 Markdown标题分割器

```python
def _markdown_header_split(self) -> List[Document]:
    """使用Markdown标题分割器进行结构化分割"""
    # 定义要分割的标题层级
    headers_to_split_on = [
        ("#", "主标题"),       # 菜品名称
        ("##", "二级标题"),    # 必备原料、计算、操作等
        ("###", "三级标题")    # 简易版本、复杂版本等
    ]

    # 创建Markdown分割器
    markdown_splitter = MarkdownHeaderTextSplitter(
        headers_to_split_on=headers_to_split_on,
        strip_headers=False  # 保留标题，便于理解上下文
    )

    all_chunks = []
    for doc in self.documents:
        # 对每个文档进行Markdown分割
        md_chunks = markdown_splitter.split_text(doc.page_content)

        # 为每个子块建立与父文档的关系
        parent_id = doc.metadata["parent_id"]

        for i, chunk in enumerate(md_chunks):
            # 为子块分配唯一ID并建立父子关系
            child_id = str(uuid.uuid4())
            chunk.metadata.update(doc.metadata)
            chunk.metadata.update({
                "chunk_id": child_id,
                "parent_id": parent_id,
                "doc_type": "child",  # 标记为子文档
                "chunk_index": i      # 在父文档中的位置
            })

            # 建立父子映射关系
            self.parent_child_map[child_id] = parent_id

        all_chunks.extend(md_chunks)

    return all_chunks
```

- **三级标题分割**: 按照`#`、`##`、`###`进行层级分割
- **保留标题**: 设置`strip_headers=False`，保留标题信息便于理解上下文
- **父子关系**: 每个子块都记录其父文档的`parent_id`
- **唯一标识**: 每个子块都有独立的`child_id`

2.3.3 分块效果示例

以"西红柿炒鸡蛋"为例，分块后的效果：

```
原文档：西红柿炒鸡蛋的做法.md (父文档)
├── 子块1：# 西红柿炒鸡蛋的做法 + 简介 + 难度评级
├── 子块2：## 必备原料和工具 + 食材清单
├── 子块3：## 计算 + 用量配比公式
├── 子块4：## 操作 + 详细制作步骤
└── 子块5：## 附加内容
```

分块逻辑:

- **子块1**: 包含一级标题及其下的所有内容（简介、难度评级），直到遇到下一个二级标题
- **子块2-5**: 每个二级标题及其下的内容形成一个独立子块
- **精确检索**: 用户问"需要什么食材"时，能精确匹配到子块2
- **上下文完整**: 生成时传递完整的父文档，包含所有必要信息

2.4 智能去重

当用户询问"宫保鸡丁怎么做"时，可能会检索到同一道菜的多个子块。我们需要智能去重，避免重复信息。

```python
def get_parent_documents(self, child_chunks: List[Document]) -> List[Document]:
    """根据子块获取对应的父文档（智能去重）"""
    # 统计每个父文档被匹配的次数（相关性指标）
    parent_relevance = {}
    parent_docs_map = {}

    # 收集所有相关的父文档ID和相关性分数
    for chunk in child_chunks:
        parent_id = chunk.metadata.get("parent_id")
        if parent_id:
            # 增加相关性计数
            parent_relevance[parent_id] = parent_relevance.get(parent_id, 0) + 1

            # 缓存父文档（避免重复查找）
            if parent_id not in parent_docs_map:
                for doc in self.documents:
                    if doc.metadata.get("parent_id") == parent_id:
                        parent_docs_map[parent_id] = doc
                        break

    # 按相关性排序并构建去重后的父文档列表
    sorted_parent_ids = sorted(parent_relevance.keys(),
                               key=lambda x: parent_relevance[x], reverse=True)

    # 构建去重后的父文档列表
    parent_docs = []
    for parent_id in sorted_parent_ids:
        if parent_id in parent_docs_map:
            parent_docs.append(parent_docs_map[parent_id])

    return parent_docs
```

去重逻辑:

1. **统计相关性**: 计算每个父文档被匹配的子块数量

2. **按相关性排序**: 匹配子块越多的菜谱排名越靠前
3. **去重输出**: 每个菜谱只输出一次完整文档